CHAPTER 12

Design A News Feed System

Show each user a ranked stream of posts from the people they follow (Facebook, Twitter, Instagram).

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a news feed: a constantly updating list of stories - status updates, photos, videos, links - from the friends and pages a user follows. Two flows: **feed publishing** (user posts -> friends' feeds are populated) and **news feed building** (aggregate friends' posts in reverse-chronological order for display).

Why interviewers ask it: The textbook fan-out problem: fan-out-on-write vs fan-out-on-read, celebrity hot-spots, caching and social graph.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Mobile, web, or both?	Both.
Features?	Publish a post, see friends' posts on the feed.
Ordering?	Reverse chronological (keep ranking simple).
Max friends?	5,000.
Traffic?	10 M DAU.
Content?	Text, images, video.

3. Requirements

Functional

- Publish a post (text/media).
- Retrieve the news feed (paginated).
- Friends' posts appear in my feed soon after posting.

Non-functional

- Low-latency feed reads (< 200-500 ms).
- High availability and scalability.
- Eventual consistency for feed contents is acceptable.

4. Back-of-the-envelope estimation

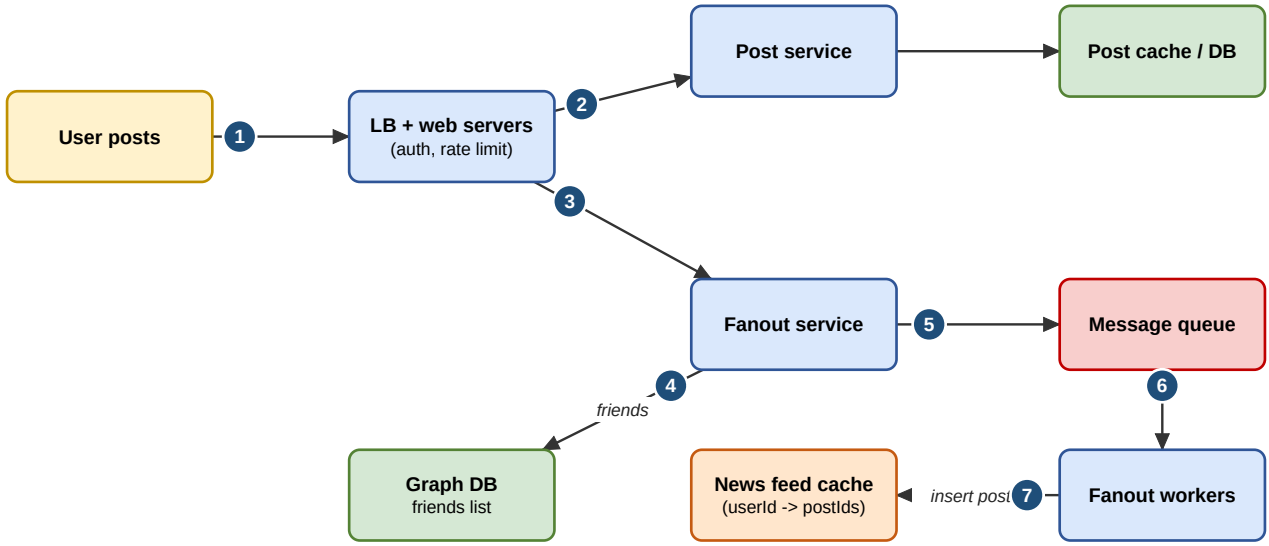
- 10 M DAU; each opens the feed about 5 times/day = 50 M feed reads/day = about **600 read QPS** (peak 2x). Posts: assume 2 posts/user/day active-posters 20% -> about 4 M posts/day = about 50 write QPS.
- Fan-out on write: 4 M posts x average 200 friends = **800 M feed inserts/day** = about 9 K/s.
- Feed cache entry = post IDs only (8 B x 500 entries x 10 M users = 40 GB) - fits in a Redis cluster.

5. API / data model

```
POST /v1/me/feed   body: { content, auth_token }       # publish a post
GET  /v1/me/feed?cursor=<post_id>&limit=20              # retrieve my news feed
```

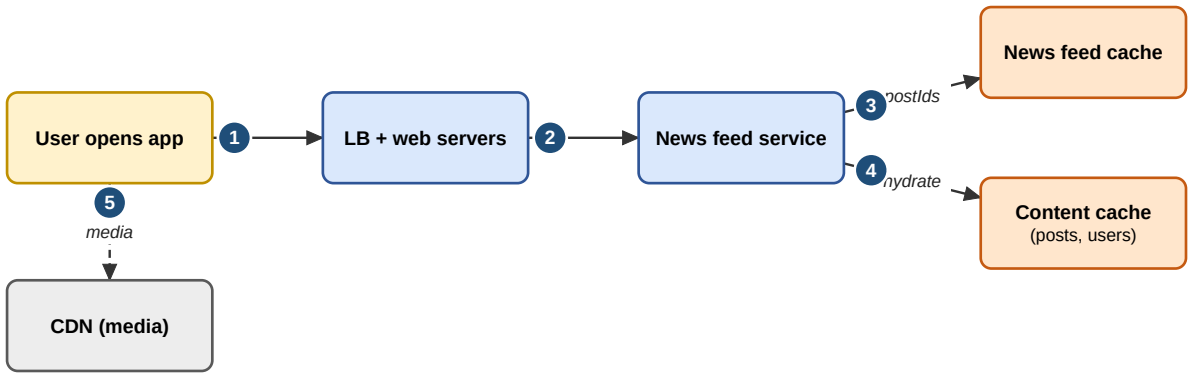
6. High-level design

Feed publishing (fan-out on write)



Step	What happens
1	A user publishes a post. The request passes through the load balancer and web servers, which **authenticate and rate-limit**.
2	The post service writes the post to the **post DB / cache**.
3	The fanout service is triggered to push the post ID into friends' feeds.
4	It fetches the author's **friend IDs** from the graph DB and filters (muted, blocked, privacy settings).
5	It enqueues one job per batch of friends on a message queue so the fan-out is asynchronous and can absorb bursts.
6	Fanout workers consume the jobs.
7	Workers insert `(post_id, user_id)` at the head of each friend's **news feed cache** (IDs only, capped at about 500-1000 entries).

News feed retrieval



Step	What happens
1	The user opens the app and requests `GET /v1/me/feed`.
2	The news feed service handles the call with the user's cursor (last seen post ID).
3	It reads the list of **post IDs** from the user's feed cache (and merges celebrity posts pulled at read time).
4	It **hydrates** the IDs into full posts, user profiles, and counters from content caches.

Step	What happens
5	The client fetches images and video directly from the **CDN**.

Request flow

- **Publish:** auth + rate limit -> save post to DB/cache -> fanout service gets friend IDs from the graph DB, filters (muted/blocked), pushes jobs to a queue -> workers prepend `(post_id, user_id)` to each friend's feed cache.
- **Read:** feed service gets a list of post IDs from the feed cache -> hydrates users/posts from content caches -> returns JSON; media served via CDN.

7. Deep dive

Fan-out on write (push model)

- Pre-compute feeds at publish time. **Pros:** feed reads are instant, since data is pre-computed. **Cons:** a user with many friends/followers triggers a huge fan-out (**hotkey problem**); wasteful for inactive users.

Fan-out on read (pull model)

- Build the feed when requested by pulling recent posts from all friends. **Pros:** no work for inactive users; no hotkey on write. **Cons:** slow reads.

Hybrid (the answer)

- Push for normal users; **pull for celebrities** (high follower count) and merge at read time. Use consistent hashing to spread load.

Cache architecture (5 layers)

- **News feed** (IDs only), **Content** (hot posts vs normal), **Social graph** (follower/following), **Action** (like/reply/etc.), **Counters** (likes, replies, followers).

Data model

- Post store (sharded, e.g. Cassandra/MySQL) keyed by post_id; user store; graph DB or adjacency tables; feed cache stores only post IDs to save memory (hydrate on read).
- Cap feed length per user (e.g. 500-1000 entries) - users rarely scroll further.

Pagination

- Use **cursor-based** pagination (last seen post_id/time) instead of offset, because the feed changes constantly.

Scaling the stack

- Keep web tier stateless; scale cache (shard); replicate DB (read replicas); queue the fan-out; use eventual consistency; scale graph storage.
- Monitor fan-out lag (time from post to appearance in feed) and queue depth.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Fan-out model	Write (push) / Read (pull) / Hybrid	Hybrid: push for regular users, pull for celebrities.
Feed storage	IDs only / full post content	IDs only: smaller memory, extra hydration hop.
Ordering	Reverse chronological / ML-ranked	Chronological is simple; ranking needs a ranking service and features.
Consistency	Strong / eventual	Eventual is fine for feeds.

9. Scaling, reliability, failure modes

- Shard feed cache by user_id; replicate for HA; rebuild from DB on cache loss.
- Fan-out workers scale horizontally; back-pressure through queue.
- Deleting posts: lazy removal at hydration time (skip missing IDs).

10. Likely follow-up questions

- How do you handle a celebrity with 50 M followers?

- How do you rank the feed with ML and still serve it fast?
- How do you handle post deletion / blocking / privacy changes?
- How would you support infinite scroll with consistent pagination?

11. Common mistakes

- Pure push or pure pull without discussing the hotkey / celebrity issue.
- Storing whole posts in each follower's feed cache (memory blow-up).
- Offset-based pagination on a changing list.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	4 min	"I'll assume web and mobile, two features - publish a post and view the news feed - with the feed in reverse chronological order, up to 5,000 friends, 10 M DAU, and text plus images and video. That is roughly 600 feed reads per second and 50 post writes per second; fan-out multiplies writes by the average friend count."
2. High-level design	10 min	"Two flows. Publishing: authenticated, rate-limited request, post saved, fanout service gets friend IDs from the graph DB, and workers push the post ID into each friend's feed cache via a queue. Retrieval: the feed service reads post IDs from the cache, hydrates them from content caches and returns the page; media comes from the CDN."
3. Deep dive	25 min	"The main choice is fan-out on write versus on read. Push gives fast reads but creates a hotkey problem for celebrities; pull is cheap on writes but slow on reads. I'd use a hybrid: push for normal users, pull for celebrities merged at read time. I'd split caching into five layers: feed, content, social graph, actions and counters, store only IDs in the feed cache, and paginate with cursors."
4. Wrap up	4 min	"The system is eventually consistent, which is fine for feeds. I'd monitor fan-out lag and queue depth. Next steps are ML ranking, handling deletes and privacy changes lazily at hydration, and sharding the feed cache by user ID."

13. 60-second recap / practice checklist

- Two flows: publish (fan-out) and retrieve (hydrate IDs).
- Hybrid fan-out; feed cache of IDs; 5 cache tiers; cursor pagination.
- Mention rate limiting, auth, queue-based fan-out, eventual consistency.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Timelines at scale (Twitter, InfoQ)
• GeeksforGeeks: Design a news feed
• Facebook: News feed ranking